

# Neuromorphic Hyperdimensional Computing for Efficiently Processing Event-Based Data

Tianyang Yu<sup>1</sup>, Graduate Student Member, IEEE, Bi Wu<sup>1</sup>, Member, IEEE, Ke Chen<sup>1</sup>, Member, IEEE, Chenggang Yan<sup>1</sup>, Member, IEEE, Gong Zhang<sup>1</sup>, Senior Member, IEEE, and Weiqiang Liu<sup>1</sup>, Senior Member, IEEE

**Abstract**—The neuromorphic sensor’s event-based data output offers significant benefits, including minimal data redundancy and exceptional time resolution, which guarantee low power consumption and heightened sensitivity during the data acquisition process. Spiking neural network (SNN), with its inherent event-driven characteristic, is well-suited for processing event-based data, and its spike-based computing mechanism enhances the efficiency of data processing. Recent studies are exploring the integration of brain-inspired hyperdimensional computing (HDC) with SNN to leverage HDC’s advantages, including the low inference and training complexity, aiming to further reduce hardware overhead associated with SNN deployment. However, existing works have not effectively harnessed the information output by SNN during hyperdimensional encoding, leading to considerable area and energy overhead. In this article, an efficient neuromorphic HDC method is proposed, featuring a simplified hyperdimensional encoding approach that considers the temporal dynamics of SNN. In addition, a lightweight accelerator design matching the proposed method is also given. Experimental results show that the proposed accelerator achieves over 50% area reduction and reduces energy consumption by 20%–90%.

**Index Terms**—Application-specific integrated circuit (ASIC), field-programmable gate array (FPGA), hyperdimensional computing (HDC), neuromorphic computing, spiking neural network (SNN).

## I. INTRODUCTION

IN THE era of the Internet of Things (IoT), edge-side devices are constrained by battery life, necessitating greater demands for extremely low energy consumption. Neuromorphic sensor, like the widely used dynamic vision sensor (DVS) [1], is well-suited for edge-side scenarios due to their exceptionally low energy consumption during the data acquisition process. In contrast to a traditional camera, DVS only captures the changes between adjacent frames, known as

event-based data, instead of all the pixels. Due to the high similarity between adjacent frames, event-based data significantly reduces redundancy, facilitating efficient recording and transmission. This reduction in redundancy further decreases the time needed for processing each frame, thus enabling DVS to sample data rapidly with very short interval. Consequently, DVS achieves high time resolution and circumvents the motion blur issue inherent in traditional camera, making event-based data contain more accurate information about motion and timing.

In terms of processing event-based data, spiking neural network (SNN) [2], as a brain-inspired neuromorphic computing paradigm, naturally aligns with event-based data due to its inherent event-driven nature. In SNN, neuron models like IF or leaky integrate-and-fire (LIF) [3] are employed to mimic the accumulation and firing of neural spike signals, akin to the processes observed in the human brain. Between neurons, binary spike (in the form of 0/1) is used to transmit information. Neurons become active only upon receiving a spike; otherwise, they remain dormant. Although the spike-based communication mechanism and event-driven nature offer high efficiency, implementing a neuron in hardware is complex as it necessitates essential components, including accumulator, registers, comparator, and reset circuit [4]. Therefore, implementing SNN with numerous neurons incurs significant area and energy overhead, making it challenging to fulfill the demands of edge side. Additionally, in edge-side scenarios requiring on-the-fly learning of new classes and samples, the cumbersome synaptic weight update workflows inherent to SNN result in a fundamental mismatch.

Recent studies [5], [6], [7] have explored integrating hyperdimensional computing (HDC), another emerging brain-inspired computational paradigm, with SNN to further enhance the efficiency of processing event-driven data while utilizing fewer neurons. HDC emulates the human brain’s method of processing information within hyperdimensional space [8], [9], encoding each input sample into a binarized hypervector with 2–10k dimensions [10], and subsequently deriving prediction results by assessing the similarity between the sample hypervector and trained class hypervectors. Since HDC primarily involves logical operations, its inherent simplicity and high parallelism enable the hybrid SNN-HDC to achieve lower complexity compared to pure SNN, which will be proved in this article. Furthermore, we also attempt to capitalize on

Received 26 January 2026; revised 3 April 2026 and 10 May 2026; accepted 12 May 2026. Date of publication 11 June 2026; date of current version 27 July 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62425404, Grant 62522406, and Grant 92364201; and in part by the Basic Research Program of Jiangsu under Grant BK20253023. (Tianyang Yu and Bi Wu contributed equally to this work.) (Corresponding author: Weiqiang Liu.)

Tianyang Yu, Bi Wu, Ke Chen, Chenggang Yan, and Weiqiang Liu are with the College of Integrated Circuits and the Key Laboratory of Aerospace Integrated Circuits and Microsystem, Ministry of Industry and Information Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China (e-mail: liuweiqiang@nuaa.edu.cn).

Gong Zhang is with the College of Electronics and Information Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China.

Digital Object Identifier 10.1109/TVLSI.2026.3695182

HDC's one-shot learning capability and streamlined retraining paradigm, so as to bolster the rapid transfer learning performance of SNN for new classes and unseen samples.

However, these existing hybrid SNN-HDC methods fall short of efficiently utilizing the information extracted from event-based data during the hyperdimensional encoding stage, which results in inevitable information loss despite their high encoding complexity. To deal with the aforementioned issues, a novel neuromorphic HDC (abbreviated as **NeuroHDC**) method is proposed, featuring an extremely simplified hyperdimensional encoding approach that specifically takes advantage of the temporal dynamics inherent in SNN. Through the thorough aggregation of spatiotemporal information derived from SNN, this encoding approach attains low complexity while ensuring almost no information loss. Simultaneously, we establish a connection between the proposed NeuroHDC and a hybrid spiking-binary neural network, and introduce a training scheme based on gradient descent algorithm. In addition, a lightweight, dedicated hardware accelerator design tailored to the proposed NeuroHDC is also proposed. This architecture achieves minimal area, inference latency, and energy consumption by leveraging time-multiplexed core modules and a stream processing workflow. Experiments demonstrate that the proposed accelerator not only achieves exceptional accuracy but also surpasses the performance of existing designs.

The main contributions are summarized as follows.

- 1) A neuromorphic HDC method with a compact hyperdimensional encoding approach is proposed. By fully harnessing the temporal dynamics of SNN, it significantly reduces the complexity associated with encoding and SNN.
- 2) For the proposed NeuroHDC, a training scheme based on a novel equivalent hybrid neural network is also proposed, which enables it to achieve state-of-the-art accuracy compared with other designs.
- 3) A lightweight hardware architecture is proposed to accelerate the proposed NeuroHDC through time-multiplexed core modules and streaming structure.

The remainder of this article is organized as follows. Section II provides the background and related work. The proposed neuromorphic HDC method and the corresponding training scheme are described in Section III. Section IV gives experimental results and comparisons of NeuroHDC. In Section V, a lightweight hardware accelerator matching the proposed method is presented. Evaluation on application-specific integrated circuit (ASIC) and FPGA and comparison with other state-of-the-art designs are given in Section VI. Finally, Section VII concludes this work.

## II. BACKGROUND AND RELATED WORK

### A. Spiking Neural Network

As the third generation of neural networks, SNN differs significantly from the second generation deep neural network (DNN) by incorporating the modeling of neuron behavior, which replaces the activation function. The neuron model introduces temporal dynamics, together with a more efficient spike-based communication and processing mechanism. To

simplify matters, the dynamics of various spiking neuron models can be described by a general formula [11] as follows:

$$\begin{aligned} V_p(t) &= f(V(t-1), X(t)) \\ S(t) &= \Theta(V_p(t) - V_{thresh}) \in \{0, 1\} \\ V(t) &= V_p(t) \times (1 - S(t)) + V_{reset} \times S(t) \end{aligned} \quad (1)$$

where  $V_p(t)$  represents the temporary membrane potential before the trigger of spike at timestep  $t$ , which is associated with the membrane potential  $V(t-1)$  at the preceding timestep, as well as the weighted postsynaptic input  $X(t)$ . Then, the neuron evaluates the membrane potential against threshold voltage  $V_{thresh}$  to determine whether to fire a spike  $S(t)$ .  $\Theta(z)$  is the Heaviside step function, whose value is 1 if  $z \geq 0$ , and 0 if  $z < 0$ . When the neuron fires a spike, its membrane potential  $V(t)$  resets to  $V_{reset}$ ; otherwise, it remains unchanged. And  $V_{reset}$  is usually set to 0 by default. It is worth noting that the primary distinction among various types of neurons is manifested in how they integrate input spikes to alter their membrane potential. For example, the integration formulas of two widely used IF and LIF neurons are presented as follows:

$$\begin{aligned} IF: V_p(t) &= V(t-1) + X(t) \\ LIF: V_p(t) &= V(t-1) + \frac{1}{\tau}(-V(t-1) + X(t)) \end{aligned} \quad (2)$$

where  $\tau$  is a constant that regulates the leakage in the LIF neuron. Considering the much lower complexity of IF neuron, which makes it more suitable for hardware implementation, this article adopts it as the default.

In terms of SNN's training, there are three categories of prevalent methods: unsupervised training, ANN-SNN conversion, and supervised training. Unsupervised training primarily relies on biologically plausible local learning rules, such as Hebbian learning [12] and spike-timing-dependent plasticity (STDP) [13]; however, it suffers from low accuracy, leading to limited application scenarios. ANN-SNN conversion uses the firing rate of each IF neuron to approximate the corresponding ReLU activation, thus converting a trained nonspiking DNN to SNN [14]. However, this kind of method cannot deal with the event-based data, which is the focus of this article, as DNN struggles to handle such data, thus rendering the foundation for conversion nonexistent. The supervised training methods employ a surrogate gradient to address the nondifferentiability issue of the step function  $\Theta(z)$  in (1), enabling SNN to be optimized by a gradient descent algorithm [15], [16] like DNN and attain comparable accuracy.

### B. Hyperdimensional Computing

Binary hypervectors with  $D_{hv}$  dimensions serve as the foundation of HDC. Thanks to the blessing of dimensionality, any two randomly generated hypervectors are approximately orthogonal with extremely high probability [9], thus enabling the representation of distinct entities or attributes. In practice, binary hypervectors can be either bipolar ( $-1/1$ ) or unipolar ( $0/1$ ). To simplify the description and software implementation, the bipolar form is preferred [17] since binarization can be readily accomplished by taking the sign of a number.

Encoding and querying are the two main processes in HDC. For the encoding process, which transforms the input sample into a hypervector, Random-Projection [18], Level-Id [19], and N-gram [20] are the three most widely used ones, which can be expressed by the following equation:

$$\begin{aligned} h &\stackrel{\text{Random}}{\underset{\text{Projection}}{=}} \text{Sign} \left( \sum_{j=1}^n f_j \times I_j \right) \\ h &\stackrel{\text{Level-Id}}{=} \text{Sign} \left( \sum_{j=1}^n L_j \otimes I_j \right) \\ h &\stackrel{\text{N-gram}}{=} \text{Sign} \left( \sum_{j=1}^n h_{j,1} \otimes \Pi_{h_{j,2}}^1 \otimes \Pi_{h_{j,3}}^2 \right). \end{aligned} \quad (3)$$

For image input,  $f_j$  denotes the  $j$ th pixel value, with  $I_j$  as its corresponding identity hypervector.  $L_j$  represents the  $j$ th pixel's level hypervector and  $\otimes$  means elementwise multiplication.  $\Pi_h^k$  denotes circular left shifting a hypervector  $h$  for  $k$  times. Both Random-Projection and Level-ID rely on randomly generated identity hypervectors (which are mutually orthogonal) to distinguish distinct elements/pixels, whereas Level-ID uses extra level hypervectors (the similarity between two level hypervectors is higher when the difference between their corresponding values is smaller) to represent each element's value. For sequence-oriented tasks, the N-gram method encodes sequential order information by means of cyclic hypervector shifting. These three methods incorporate the fundamental encoding operations of binding ( $\times$  or  $\otimes$ ), bundling ( $\sum$ ), and circular shifting ( $\Pi$ ), upon which the other encoding schemes can be founded.

By contrast, the querying process is relatively unified, that is, to calculate the similarity between the sample hypervector and the trained class hypervectors and then select the most similar class as prediction. Therefore, the simplicity of HDC endows it with a distinct energy efficiency advantage for edge-side classification and clustering tasks [21], [22], [23].

The fundamental training scheme of HDC involves encoding samples from the same class within the training set into hypervectors, summing these hypervectors, and subsequently doing binarization to obtain class hypervectors. Based on this, the retraining scheme [24] that fine-tunes class hypervectors to mitigate classification errors further enhances HDC's accuracy. The latest works [10], [25] establish a link between the querying process and binary neural network (BNN), employing the gradient descent algorithm to update class hypervectors, which leads to a substantial improvement in accuracy.

With respect to the event-based data addressed in this article, EventHD [26] employs pure HDC for data processing, which gives two encoding schemes as follows:

$$\begin{aligned} h_t &= \sum_{j=1}^n P_j \otimes I_j, \quad h = h_1 + \Pi_{h_2}^1 + \dots + \Pi_{h_T}^{T-1} \\ h_t &= \sum_{j=1}^n P_j \otimes I_j, \quad h = \sum_{t=1}^T (h_t \otimes B_t). \end{aligned} \quad (4)$$

For them, hyperdimensional encoding is first applied to each frame (as defined in Section III-B) of the event-based data.  $I_j$  denotes the identity hypervector corresponding to the  $j$ th pixel in the frame, whereas  $P_j$  characterizes its polarity. It is worth noting that, to capture spatial correlation, EventHD constructs a mechanism to enhance the similarity among  $I_j$  corresponding to spatially neighboring pixels, instead of orthogonality. Then, frame hypervector fusion can be achieved through two schemes: post-shift bundling and post-binding bundling. In post-binding bundling,  $B_t$  represents the hypervector corresponding to the  $t$ th timestep, which is also specially constructed to capture spatial correlations. In addition, event-based HDC has also been deployed in multiple real-world applications: drone control [27], object pose estimation [28], and robotic grasping perception [29].

### C. Integration of SNN and HDC

Considering the hardware deployment complexity of SNN and HDC's limitation in handling temporal dynamics of event-based data [5], efforts have been initiated toward integrating SNN and HDC [5], [6], [7], [30]. In these works, event-based data are first processed by SNN (multilayer [30] or single layer [5], [6], [7]); then, the extracted information undergoes hyperdimensional encoding, and finally, the classification result is derived by querying within the hyperdimensional space. Among them, HyperSpikeASIC [6] and Spiking-HDC [7] particularly stand out as they conducted a comprehensive evaluation on ASIC, surpassing the mere algorithm-level assessments conducted in other works. Recent work [31] provides further support for the on-chip training of Spiking-HDC. However, the aforementioned works all suffer from information loss and high complexity during hyperdimensional encoding, which will be analyzed in Section III-A.

Other works like HyperEncoding [32] and SynapseHD [33] also explore the integration of SNN and HDC, but they focus on images. HyperEncoding employs hyperdimensional encoding to convert a raw image into hypervector and then feeds it into SNN, which effectively bypasses the intensive floating-point operations, thus yielding a substantial reduction in computational cost. While SynapseHD aims to utilize HDC's gradient-free training method to reduce SNN's training cost. Furthermore, HDC can be utilized to decode SNN outputs [34], where mutually orthogonal hypervectors substitute the one-hot labels used in existing SNN training schemes. Due to fundamentally different goal and motivation, these works fall outside the scope of our comparative analysis.

## III. PROPOSED NEUROMORPHIC HDC METHOD

### A. Analysis and the Proposed Encoding Approach

Both HyperSpikeASIC and Spiking-HDC utilize a one-layer SNN to extract information from event-based data and carry out HDC's encoding and querying. Specifically, HyperSpikeASIC [6] takes all the neurons' membrane

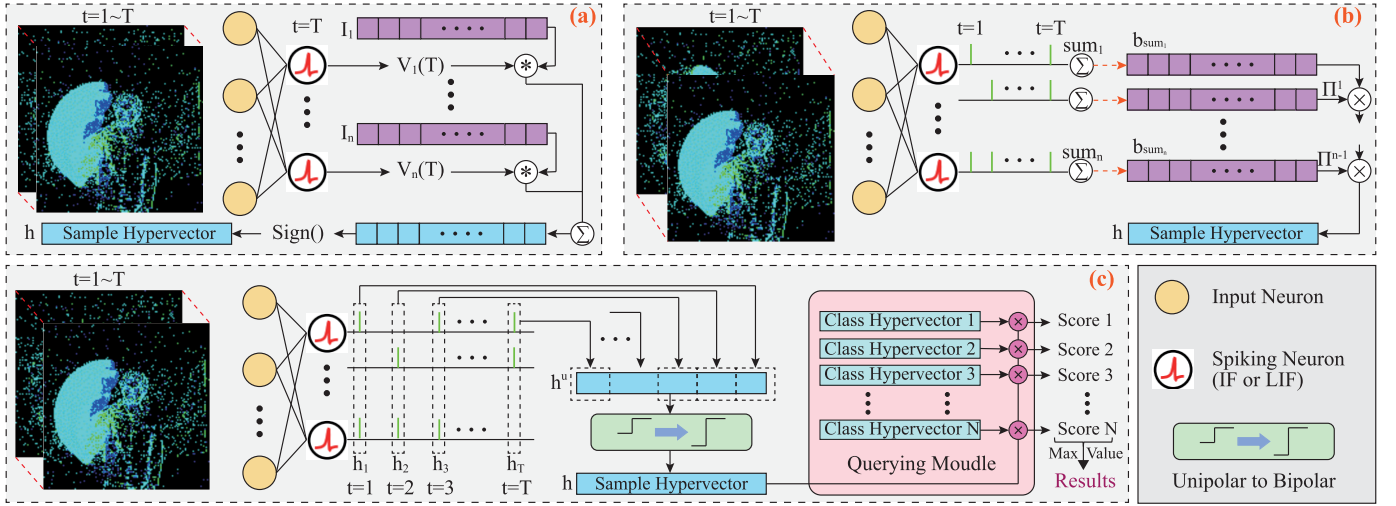


Fig. 1. Hyperdimensional encoding approach in (a) HyperSpikeASIC, (b) spiking-HDC, and (c) overall workflow of the proposed NeuroHDC.

potential as extracted information, as shown in Fig. 1(a). The encoding process can be expressed as follows:

$$h = \text{Sign} \left( \sum_{j=1}^n V_j(T) \times I_j \right) \in \{-1, 1\}^{D_{nv}} \quad (5)$$

where  $n$  is the number of spiking neurons,  $T$  is the total timesteps, and  $V_j(T)$  is the membrane potential of the  $j$ th neuron at the last timestep.  $I_j \in \{-1, 1\}^{D_{nv}}$  is the  $j$ th identity hypervector, which is randomly chosen to distinguish the  $j$ th neuron with other neurons, since randomly generated hypervectors are orthogonal to one another. The use of  $\text{Sign}()$  function in this Random-Projection encoding approach leads to a loss of information contained in the membrane potential. Spiking-HDC [7] aggregates the output spikes from each neuron at every timestep and encodes the accumulated results, as shown in Fig. 1(b). There are three steps in the encoding, which can be outlined in detail as follows:

$$\begin{aligned} \text{sum}_j &= \sum_{t=1}^T S_j(t) \in [0, T], h_j = b_{\text{sum}_j} \\ h &= h_1 \otimes \Pi_{h_2}^1 \otimes \Pi_{h_3}^2 \otimes \dots \otimes \Pi_{h_n}^{n-1} \end{aligned} \quad (6)$$

where  $\text{sum}_j$  is the accumulated results corresponding to the  $j$ th neuron.  $b_0, b_1, \dots, b_T \in \{-1, 1\}^{D_{nv}}$  are randomly generated hypervectors to represent  $T + 1$  possible accumulation values.  $\Pi_h^k$  denotes circular left shifting a hypervector  $h$  for  $k$  times. In this modified N-gram encoding approach, the continuous elementwise multiplication operations  $\otimes$  lead to unavoidable confusion during information aggregation, thus resulting in information loss.

According to the above analysis, previous works all employ variations of existing hyperdimensional encoding approaches; however, they do not effectively align with the characteristics of SNN and also lead to information loss. Since the input for encoding is the valid information extracted by SNN, the aforementioned loss necessitates that SNN supplies more information to compensate. Consequently, this results in increased complexity, evident in a greater number of neurons, which will be discussed in the subsequent section.

In this case, a novel hyperdimensional encoding approach matching SNN is proposed to avoid these problems, as shown in Fig. 1(c), which can be expressed as follows:

$$\begin{aligned} h_t &= [S_1(t), S_2(t), \dots, S_j(t), \dots, S_n(t)] \\ h^u &= [h_1, h_2, \dots, h_t, \dots, h_T], h = h^u \times 2 - 1 \end{aligned} \quad (7)$$

where the spikes emitted by all neurons across all timesteps are concatenated to create the hypervector. Bipolar hypervector offers distinct advantages in software implementation (as outlined in Section II-B), while the spike  $S_j(t)$  is unipolar. This discrepancy makes it essential to transform the unipolar hypervector  $h^u$  obtained from direct concatenation. However, this conversion is bypassed for hardware implementation, where the unipolar hypervector is used. The proposed method efficiently harnesses all the spikes without any loss of information, while eliminating the need for pregenerated hypervectors and complicated operations.

### B. Overall Workflow

The proposed NeuroHDC contains a single-layer fully connected SNN, hyperdimensional encoding, and querying.

To facilitate training and simulation at the software level, the event-based data can be expressed in the form of a tensor. As illustrated in Fig. 1(c), the input  $D$  comprises  $T$  frames, each corresponding to a distinct timestep. Each frame  $D(t)$  has  $P$  channels;  $P$  is typically set to 2, representing the two scenarios where the pixel brightness either increases or decreases. While the original event-based data are in the form of quadruple  $(t', p, x, y)$ . In each quadruple,  $t'$  is the real-time timestamp captured by DVS and  $(x, y)$  represents the coordinate where the brightness change occurs;  $p$  stands for polarity, with a value of 0 signifying a decrease in brightness, whereas 1 denotes an increase. In this case, to convert the data from its original quadruple form into the tensor form, it is necessary to aggregate all events within each timestep

$$E_{(p,x,y)}(p_k, x_k, y_k) = \begin{cases} 1 & (p, x, y) = (p_k, x_k, y_k) \\ 0 & (p, x, y) \neq (p_k, x_k, y_k) \end{cases}$$

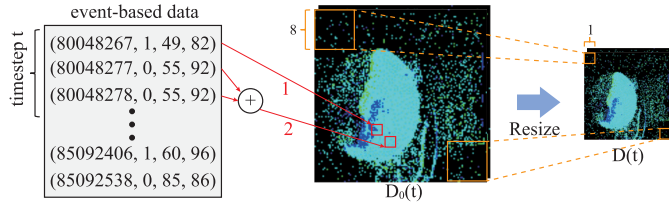


Fig. 2. Example of event-based data conversion into a frame.

$$D_0(t)(p, x, y) = \sum_{t'_k \in t} E_{(p, x, y)}(p_k, x_k, y_k) \quad (8)$$

where  $(t'_k, p_k, x_k, y_k)$  means the  $k$ th event. To simplify the process of dividing all events into  $T$  timesteps, we divide them uniformly, ensuring an equal number of events in each. The left half of Fig. 2 is an illustrative representation of (8). The more events that occur at the same coordinate, the greater the value of the corresponding pixel point in  $D_0(t)$ . The parameter count of the SNN is directly influenced by the size of  $D_0(t)$ , as a larger size necessitates more input neurons and results in a higher parameter burden. To mitigate this overhead, we resize  $D_0(t)$  using “SumPool,” which sums pixels within nonoverlapping windows, as illustrated in the right half of Fig. 2. The resized frame is  $D(t) \in \mathbb{R}^{P \times H \times W}$ , where  $H$  and  $W$  represent its height and width, respectively.

By sequentially flattening each frame ( $\mathbb{R}^{P \times H \times W} \rightarrow \mathbb{R}^{(P \cdot H \cdot W)}$ ) and feeding it into the SNN’s input neurons, we can obtain the output spikes from the  $n$  spiking neurons. These spikes are then encoded using (7) to generate the sample hypervector  $h \in \{-1, 1\}^{1 \times D_{hv}}$ . Note that the hypervector dimension  $D_{hv}$  is governed by the total timesteps  $T$  and the spiking neuron number  $n$  as shown in (7), the specific settings of which will be detailed in Section IV-B. Ultimately,  $h$  is compared against the trained class hypervectors  $C$ , and the class associated with the most similar class hypervector is selected as the prediction, which can be summarized as follows:

$$\begin{aligned} Score_i &= \sum_{m=1}^{D_{hv}} h(m) \times C_i(m) \\ Pred &= \text{Argmax}\{Score_1, Score_2, \dots, Score_N\} \end{aligned} \quad (9)$$

where  $C_i$  means the  $i$ th class hypervector and  $N$  is the number of classes. Notably, we employ the inner product rather than cosine similarity to denote each  $Score_i$ . Since for bipolar hypervectors, the outcome of inner product, when multiplied by a constant coefficient, equates to the cosine similarity.

### C. Training Scheme

As mentioned in Section II, HDC’s querying process can be analogous to a single-layer BNN [25], and the gradient descent algorithm is utilized to train the class hypervectors, thereby enhancing HDC’s accuracy. Based on this, thanks to the differentiability of the proposed encoding approach, we establish an equivalence between NeuroHDC and a hybrid spiking-binary neural network, facilitating joint training and optimization of both SNN and HDC. We will introduce the training process of HDC’s querying and SNN in the order of backpropagation.

Since class hypervectors  $C \in \{-1, 1\}^{N \times D_{hv}}$  are binarized while gradients are represented as floating-point numbers, we incorporate a matrix  $W^c$ , which shares the same shape as the class hypervectors, to accumulate the gradients. Class hypervectors can be obtained by binarizing  $W^c$ , and subsequently employed to compute the similarity score in the querying process during inference

$$Score = C \cdot h^T = \text{Sign}(W^c) \cdot h^T \quad (10)$$

where  $\cdot$  means matrix multiplication,  $h \in \{-1, 1\}^{1 \times D_{hv}}$ , and  $Score \in \mathbb{R}^{N \times 1}$ . The nondifferentiability of the  $\text{Sign}()$  function is solved by gradient approximation, consistent with previous work [10], and thus,  $W^c$  can be updated through gradients.

For the proposed encoding approach, the computation of the gradient is straightforward:

$$h = h^u \times 2 - 1 \rightarrow \frac{\partial L}{\partial h^u} = \frac{1}{2} \frac{\partial L}{\partial h} = \frac{1}{2} \left( \frac{\partial L}{\partial Score} \right)^T \cdot C \quad (11)$$

where  $L$  is the loss function, which is the cross entropy between  $Score$  and the ground-truth label.

When calculating gradients related to spiking neurons in SNN, it is essential to consider both temporal and spatial characteristics. This encompasses the consideration of gradients that propagate through timesteps, together with those conveyed along spike, which can be formulated as follows:

$$\frac{\partial L}{\partial V_p(t)} = \frac{\partial L}{\partial V_p(t+1)} \frac{\partial V_p(t+1)}{\partial V_p(t)} + \frac{\partial L}{\partial S(t)} \frac{\partial S(t)}{\partial V_p(t)} \quad (12)$$

$$\frac{\partial V_p(t+1)}{\partial V_p(t)} = \frac{\partial V_p(t+1)}{\partial V(t)} \frac{\partial V(t)}{\partial V_p(t)} \quad (13)$$

$$\frac{\partial L}{\partial S(t)} = \frac{\partial L}{\partial h^u} ((t-1) \times n : t \times n) \quad (14)$$

where the first term in (12) is the temporal gradient and the second term is the spatial gradient. As evident from the first term, the gradient at any timestep is contingent upon the gradient at the subsequent timestep. Notably, the gradient at the final timestep lacks the temporal term:

$$\frac{\partial L}{\partial V_p(T)} = \frac{\partial L}{\partial S(T)} \frac{\partial S(T)}{\partial V_p(T)}. \quad (15)$$

Since the partial derivatives like  $(\partial V_p(t+1))/\partial V(t)$  and  $\partial V(t)/\partial V_p(t)$ ,  $\partial S(t)/\partial V_p(t)$  can be readily calculated according to (1), the details of which are omitted here. It is important to highlight that the surrogate gradient method [35] is essential for addressing the issue of nondifferentiability arising from the  $\Theta()$  function during the calculation of  $\partial S(t)/\partial V_p(t)$ . Based on (12)–(15) and the preceding analysis, the gradient  $\partial L/\partial V_p(t)$  at any imestep can be obtained. Furthermore, the gradient associated with SNN’s weight  $W^s \in \mathbb{R}^{n \times (P \cdot H \cdot W)}$  can be derived

$$\begin{aligned} \frac{\partial L}{\partial W^s} &= \sum_{t=1}^T \left( \frac{\partial L}{\partial V_p(t)} \frac{\partial V_p(t)}{\partial X(t)} \frac{\partial X(t)}{\partial W^s} \right) \\ X(t) &= W^s \cdot D_{flat}(t)^T \rightarrow \frac{\partial X(t)}{\partial W^s} = D_{flat}(t)^T \end{aligned} \quad (16)$$

where  $\partial V_p(t)/\partial X(t)$  can be calculated according to (1) and  $D_{flat}(t) \in \mathbb{R}^{1 \times (P \cdot H \cdot W)}$  is the flattened input tensor. As a result, SNN’s weight  $W^s$  can be jointly optimized with HDC’s class hypervectors through the gradient descent algorithm.

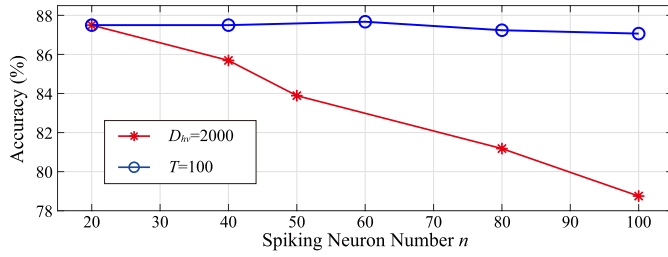


Fig. 3. Accuracy curve as spiking neuron number  $n$  scaling on DVS-Gesture.

In addition, to reduce the storage and computational requirements of the SNN part within the proposed NeuroHDC, we quantize the floating-point weight  $W^s$  to the type of 8-bit signed integer. Through the incorporation of quantization-aware training (QAT) scheme [36] for  $W^s$  into the proposed training scheme, quantization of  $W^s$  can be achieved concurrently during the training process.

#### IV. EXPERIMENTS

##### A. Dataset and Experimental Setup

Three of the most widely used event-based datasets are used to evaluate the effectiveness of the proposed NeuroHDC:

*N-MNIST* [37] is derived from the MNIST dataset by employing a moving DVS to record MNIST samples displayed on a static screen. This approach mimics the saccadic movements found in biological visual systems, effectively capturing the temporal-spatio dynamics of handwritten digits.

*DVS-Gesture* [38] comprises gestures captured from 29 volunteers under three distinct lighting conditions using DVS. It encompasses 11 gesture classes, including actions like “Hand Clapping” and “Waving the Right Arm Clockwise.” The 11th class, labeled as “Other,” contains samples not fitting into the first 10 classes; thus, following previous studies [7], our experiments exclude this class.

*DVS-ASL* [39] contains 24 gesture classes, each corresponding to one of the 24 letters in American Sign Language (ASL). This dataset is randomly split with 80% of the data for training and the remaining 20% for testing.

In subsequent experiments, unless otherwise specified, the dimension  $D_{hv}$  of the hypervector is set to  $2K$  by default, and SNN is configured with 512 input neurons, maintaining consistency with Spiking-HDC [7]. This means that each frame is in the form of  $\mathbb{R}^{2 \times 16 \times 16}$  during training. The total timesteps  $T$  is constrained to a maximum of 100, as per settings in HyperSpikeASIC [6]. For the training of NeuroHDC, the Adam optimizer is employed for better convergence.

##### B. Case Study

1) *Spiking Neuron Number and Timestep Number*: The proposed hyperdimensional encoding approach ensures that the spiking neuron number  $n$  is inversely proportional to the timestep number  $T$ . This is because it concatenates the output spikes across all timesteps to generate the sample hypervector, where  $D_{hv} = n \times T$ . In this case, we examine multiple  $(n, T)$  pairs, with the corresponding results presented in Fig. 3. As indicated by the results depicted by the red line in the figure,

TABLE I  
ACCURACY WITH TIMESTEP SCALING

|             | $T = 100$<br>$D_{hv} = 2000$ | $T = 150$<br>$D_{hv} = 3000$ | $T = 200$<br>$D_{hv} = 4000$ |
|-------------|------------------------------|------------------------------|------------------------------|
| DVS-Gesture | 87.5%                        | 87.8%                        | 87.88%                       |
| N-MNIST     | 97.28%                       | 97.38%                       | 97.36%                       |

TABLE II  
ACCURACY COMPARISON BETWEEN NEUROHDC AND  
OTHER STATE-OF-THE-ART DESIGNS

|                                 | Spiking<br>Neuron<br>Number $n$ | N-MNIST       | DVS-Gesture  | DVS-ASL       |
|---------------------------------|---------------------------------|---------------|--------------|---------------|
| HyperSpikeASIC <sup>a</sup> [6] | >>200                           | 95.2%         | 85.3%        | 87.8%         |
| Spiking-HDC <sup>b</sup> [7]    | 200                             | 94.8%         | 87.1%        | -             |
| Ours NeuroHDC <sup>c</sup>      | <b>20/40</b>                    | <b>97.28%</b> | <b>87.5%</b> | <b>89.38%</b> |

<sup>a</sup>  $D_{hv} = 4K$ . <sup>b</sup>  $D_{hv} = 2K$ .

<sup>c</sup> For N-MNIST and DVS-Gesture datasets,  $n = 20$ , thus  $D_{hv} = 2K$ . For more complicated DVS-ASL dataset,  $n = 40$ , thus  $D_{hv} = 4K$ .

for a fixed  $D_{hv}$ , an increase in  $n$  corresponds to a decrease in  $T$ , which subsequently leads to a decline in accuracy. This occurs because a smaller  $T$  renders spiking neurons less effective at capturing the temporal dynamics inherent in event-based data, and even a greater number of spiking neurons still cannot compensate for this. Thus, it is advisable to set  $T$  to 100. While results depicted by the blue line in Fig. 3 indicate that, when  $T = 100$ , increasing spiking neuron number  $n$  does not lead to a substantial improvement in accuracy, but rather results in a higher hardware overhead to implement these neurons. Therefore, we suggest setting  $n$  to 20 by default.

2) *Scalability of Timesteps*: Building upon the previous analysis, we further examine how scaling the timestep number  $T$  affects accuracy when the spiking neuron number  $n$  is held constant. As shown in Table I, increasing the timestep number yields almost no accuracy gain, indicating that 100 timesteps are sufficient to capture the necessary temporal information.

##### C. Comparison With State-of-the-Art Algorithms

Accuracy comparison between the proposed NeuroHDC and other state-of-the-art designs is presented in Table II. It can be seen that NeuroHDC achieves comparable or even higher accuracy, even when using fewer spiking neurons and hypervectors with lower  $D_{hv}$ . Two primary reasons underlie this: NeuroHDC optimally exploits spatial-temporal information extracted by neurons, and it prevents information loss during the encoding process.

HyperSpikeASIC uses the membrane potential of neurons at the final timestep, which prevents it from integrating temporal dynamics into subsequent steps; and the *Sign()* operation in its encoding process causes information loss. Spiking-HDC’s continuous elementwise multiplication of hypervectors introduces confusion, which also results in information loss; furthermore, due to the nondifferentiable nature of its encoding approach, it has to train the SNN and HDC parts independently during training, which is obviously suboptimal. Conversely,

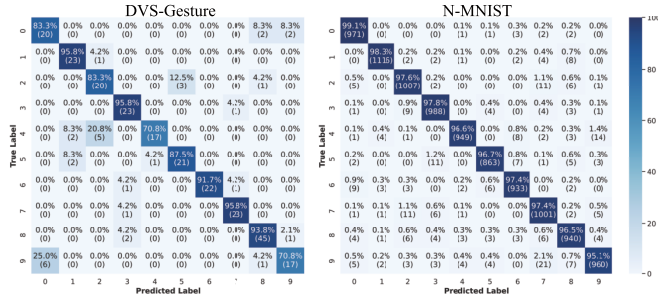


Fig. 4. Confusion matrices of DVS-Gesture and N-MNIST datasets.

owing to its differentiability, the proposed encoding approach facilitates the comprehensive and synergistic optimization of NeuroHDC through the proposed training scheme in Section III-C.

Fig. 4 presents the confusion matrices for the DVS-Gesture and N-MNIST datasets. For DVS-Gesture, the dominant misclassifications are observed when the Air Guitar gesture (Label 9) is mistaken for Hand Clapping (Label 0), and when the Right Arm Counterclockwise gesture (Label 4) is misidentified as Left Hand Wave (Label 2). This indicates that while the model may slightly confuse gestures with overlapping spatiotemporal patterns, it effectively avoids severe errors across distinct classes. Furthermore, for the N-MNIST dataset, where interclass differences are more pronounced, such confusions are nearly negligible.

#### D. Transfer Learning Performance

Given that practical applications may involve the need for rapid training of additional classes on the edge-side, the concise training scheme of HDC is inherently well-suited to address this need:

$$C_i^{nb} = \sum_{h \in \phi_i} h, \quad C_i = \text{Sign}(C_i^{nb}) \quad (17)$$

where  $\phi_i$  is the sample hypervector set of the  $i$ th class. To enhance this single-shot training scheme, it is often necessary to perform fine-tuning, which is called retraining [24]

$$\begin{aligned} C_{match}^{nb} &= C_{match}^{nb} + \alpha \times h \\ C_{mismatch}^{nb} &= C_{mismatch}^{nb} - \alpha \times h \end{aligned} \quad (18)$$

where  $C_{match}^{nb}$  is the one corresponding to the correct class and  $C_{mismatch}^{nb}$  corresponds to the prediction, when wrong prediction occurs.  $\alpha$  denotes the learning rate, which governs the update speed of class hypervectors and is usually confined from 0 to 1. We conduct experiments to evaluate the transfer learning capability of the proposed NeuroHDC. Specifically, we pretrain NeuroHDC using samples from  $N_p$  classes (source domain) through the proposed training scheme. Then, we employ the aforementioned one-shot learning and retraining, using samples from  $N_p$  classes (source domain) and the additional  $N - N_p$  new classes (target domain). Results in Fig. 5 show that NeuroHDC exhibits exceptional transfer learning capability in various datasets, effectively harnessing the efficient training attribute of HDC, while only experiencing an acceptable

accuracy decline when compared to the proposed training scheme. Certainly, under normal circumstances, the proposed training scheme is always preferred to be used, due to its superior accuracy and the fact that its higher training overhead does not have any influence on inference.

#### E. Discussion

The proposed NeuroHDC serves as a unified dual-mode framework. The first mode functions as an integration of SNN and BNN, while the second mode operates as an SNN coupled with HDC. Notably, these two modes become functionally identical during inference, as the querying process of HDC is computationally equivalent to the forward pass of a binary fully connected layer. The primary objective of Mode 1 is to enable effective SNN pretraining. By leveraging our differentiable encoding method, NeuroHDC is trainable via gradient descent (as shown in Section III-C), allowing the SNN representations to align seamlessly with the hyperdimensional space. Moreover, the lossless nature of this encoding method preserves feature integrity, which is a crucial factor behind the superiority of NeuroHDC. Building upon this foundation, Mode 2 is specifically designed for efficient transfer learning (as shown in Section IV-D). It inherits the pretrained SNN from Mode 1 and exploits the low-overhead training and rapid fine-tuning capabilities of HDC.

### V. NEUROHDC-BASED ACCELERATOR

#### A. Overall Architecture

As shown in Fig. 6, the proposed accelerator primarily comprises: one feature extractor, which includes  $n$  IF neuron modules;  $N_{max}$  hyperdimensional querying modules; and several small modules to control or assist these two major parts.

Each IF neuron module encompasses the weight of the synapses connected to the neuron, in addition to the register, adder, comparator, and reset logic that are directly associated with the neuron's functionality. Given that there are 512 input neurons, one neuron's weight requires  $512 \times 8$ -bit of space to be stored on-chip. Due to the specification constraints of the available SRAM IP, we utilize a  $512 \times 32$ -bit SRAM macro to store the weights of four neurons. The bitwidth of register&adder is set to 22, which is adequate to preclude any overflow of the membrane potential during accumulation.

Each hyperdimensional querying module is associated with a specific class; thus, it needs to contain the corresponding class hypervector. Considering that  $D_{hv} = 2K$  and the spiking neuron number  $n$  is 20, the SRAM for the class hypervector should have the capacity of  $100 \times 20$ -bit. Owing to the available SRAM IP's specification constraints, we are compelled to employ a  $128 \times 24$ -bit SRAM macro, with the last 28 rows and the last 4 columns remaining unused. Furthermore, there are  $n$  XNOR gates, an  $n$ -bit register, and an  $n$ -bit input accumulator in this module, which can be time-multiplexed among timesteps. It is worth noting that any alteration in the values of  $D_{hv}$  and  $n$  necessitates adjustments to both the SRAM macro specification and the aforementioned circuits.

The whole accelerator is governed by three configuration registers within the register group, which store operating mode,

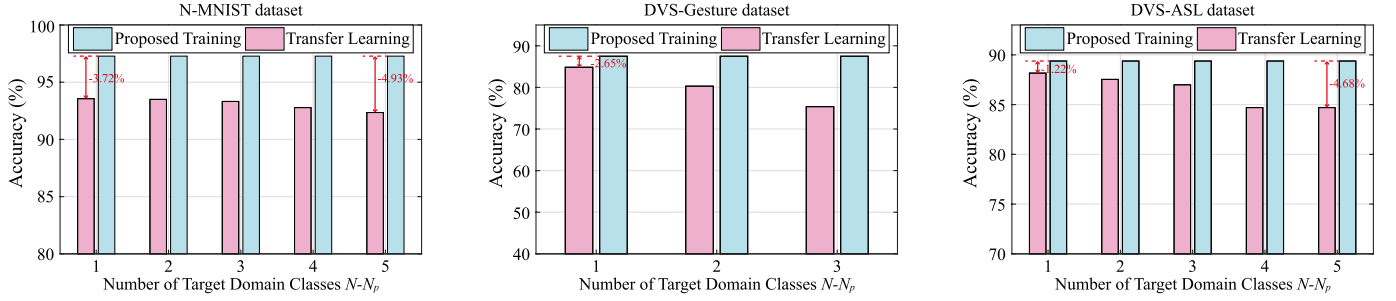


Fig. 5. Transfer learning performance on various datasets.

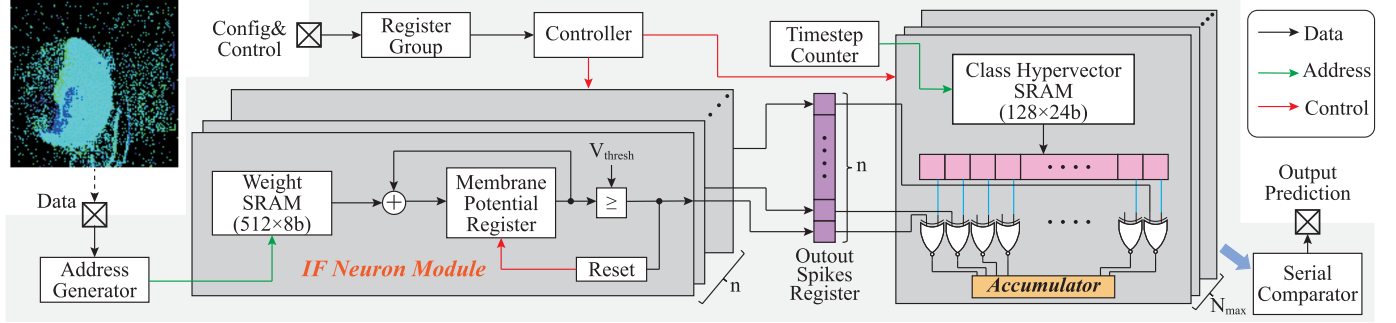


Fig. 6. Architecture of the proposed NeuroHDC-based accelerator.

$V_{thresh}$ , the number of events  $N_e$  in one timestep, and the actual class number  $N$  ( $N < N_{max}$ ), respectively. Among them, the operating mode has three states: idle, data (SNN's weight and class hypervectors) writing, and execution.  $V_{thresh}$  connects to the comparator in each IF neuron module.  $N_e$  connects to the timestep count module.  $N$  specifies the number of hyperdimensional querying modules that are actively utilized.

### B. Workflow and Details

Prior to the accelerator's normal execution, a data writing process must be carried out, wherein the SNN's weight and class hypervectors are sequentially transferred to the respective SRAM macro via the Data interface. Typically, this process is carried out just once and is repeated only when the accelerator requires reconfiguration for a new task.

When the accelerator is in the execution mode, event-based data is continuously streamed in through the data interface. It is noteworthy that, in contrast to the aforementioned training process, which reconstructs event-based data into  $T$  frames, the original address-event representation (AER) format of event-based data is preserved, thereby upholding the inherent characteristics of sparsity and efficiency. Upon the arrival of an event ( $t', p, x, y$ ), the address generator determines the address  $addr$  of the corresponding input neuron by leveraging the address ( $x, y$ ) and polarity  $p$  within the event. Given the 512 input neurons and the frame format  $\mathbb{R}^{2 \times 16 \times 16}$  detailed in Section IV-A, the aforementioned address calculation can be formulated as follows:

$$addr = p \times 256 + \lfloor y/\beta \rfloor \times 16 + \lfloor x/\beta \rfloor \quad (19)$$

where  $\beta$  is the resize factor, which is determined by the specific dataset, and  $\lfloor \cdot \rfloor$  denotes the floor function. Then, each IF neuron

module identifies the relevant weight within its SRAM macro according to the input neuron address, and integrates it into the membrane potential, as detailed next

$$R_j^V = R_j^V + \sum W_j^s(addr) \quad (20)$$

where  $R_j^V$  is the membrane potential register in the  $j$ th IF neuron module and  $W_j^s$  means the weight corresponding to the  $j$ th IF neuron. Meanwhile, the event counter within the timestep counter module is incremented by one.

As mentioned in Section III-B, we evenly divide all the events into  $T$  timesteps, meaning that each timestep contains  $N_e$  events. Therefore, when the event counter reaches  $N_e$ , it indicates that the events within a timestep have been processed. At this time, three steps need to be done in sequence. First, each IF neuron module compares the membrane potential  $R_j^V$  with  $V_{thresh}$  to decide whether to generate a spike and records the outcome in the output spikes register; if a spike is emitted, the membrane potential register resets to 0. Then, the event counter is reset, and the time step counter is incremented by one. Subsequently, according to the value of timestep counter, each hyperdimensional query module reads the corresponding  $n$ -bit segment from the class hypervector SRAM into the register, which is compared bitwise with the elements in the output spikes register using  $n$  XNOR gates. The comparison results are fed into the accumulator. After  $T$  timesteps have elapsed, the value stored in the accumulator represents the similarity  $Score_i$ . The aforementioned process can be mathematically expressed as follows:

$$Score_i = \sum_{t=1}^T \sum_{m=(n-1) \times T}^{n \times T} (h^u(m) \text{ XNOR } C_i^u(m)) \quad (21)$$

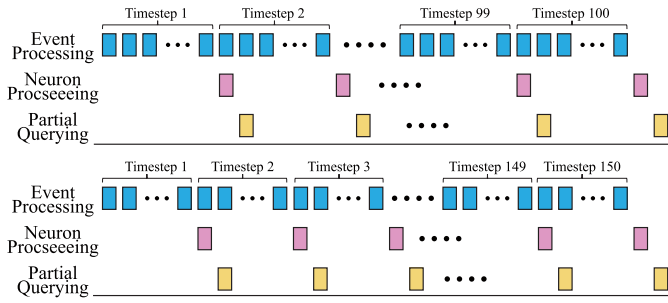


Fig. 7. Stream processing procedures at different timesteps.

where  $C_i^u$  is the unipolar form of the  $i$ th class hypervector, and we use the unipolar sample hypervector  $h^u$  without transformation since signals within digital circuits exist in unipolar form. Once the  $N$  similarity values obtained previously are sequentially input into the serial comparator, the ultimate output prediction is generated.

### C. Stream Processing and Analysis

As illustrated in Fig. 7, the proposed NeuroHDC-based accelerator processes incoming events in a streaming fashion, dynamically updating the membrane potential using the event data and SNN's weight. As discussed in Section III-B, the number of timesteps only affects the granularity of event partitioning without changing the total event count. Consequently, it does not impact the latency of the accelerator. However, increasing the number of timesteps expands the hypervector's dimension, which necessitates larger SRAM capacity and consequently increases the area and energy consumption of the accelerator. Furthermore, since timestep number  $T$  exceeding 100 yields no further improvements in accuracy (Section IV-B2), the default configuration for the proposed accelerator is fixed at  $T = 100$ .

## VI. HARDWARE EVALUATION

### A. Experimental Setup

To evaluate the performance of NeuroHDC-based accelerator with 100-MHz main frequency, we first implement it using Verilog HDL. Then, Synopsys VCS is used to simulate its functionality, and Synopsys Verdi is used to check the wave. Subsequently, we use Yosys with Skywater 130-nm cell library and SRAM IP to synthesize it and OpenRoad for the physical design (including floorplan, placement, clock tree synthesis, routing, etc). Finally, Synopsys PrimeTime in time-based mode is employed to analyze the energy consumption.

For the evaluation on FPGA, the entire design process can be completed within Xilinx Vivado 2022.1, with steps of synthesis, implementation, and bitstream generation. We use the built-in power estimation tool in Vivado to evaluate the accelerator's power and the Ultra96v2 development board as the deployment platform to measure its throughput.

### B. Evaluation on ASIC

Table III offers a comparative analysis of the performance between the proposed accelerator and other state-of-the-art

designs. Considering that the proposed NeuroHDC necessitates distinct hyperparameter settings to accommodate varying levels of dataset complexity, two accelerator specifications (small and large) are given. In addition, the proposed designs' area and energy consumption are scaled to the 45-nm node according to [40], ensuring a relatively fair comparison with other works. The comparison counterparts are implemented under 28- and 40-nm processes, whereas the proposed design is scaled to the 45nm node, as the data presented in [40] does not support direct scaling from the 130-nm node to 28-/40-nm node. This setup effectively demonstrates the merits of our design—delivering superior performance on a less advanced technology node further underscores its advantage.

HyperSpikeASIC [6] and Spiking-HDC [7] share the closest affinity with this article's goal of merging SNN and HDC for event-based data processing. Both also provide ASIC designs and evaluations, thus serving as suitable comparison groups. HyperSpikeASIC's SNN part comprises two core modules: synapse state update and neuron state update. The membrane potential registers in the neuron state update module interface with the subsequent HDC part, which integrates encoding and querying circuits. Spiking-HDC employs a compact design by integrating the synapse state update function into each spiking neuron module. It interfaces the SNN part with the HDC part via spike accumulation register.

Results show that the proposed accelerators achieve over 50% area savings; and they reduce energy consumption by more than 20%, with the maximum reduction exceeding 90%. We ascribe this outcome to the remarkable effectiveness and efficiency of the proposed encoding approach, along with the substantial decrease in neuron count that it brings about. HyperSpikeASIC takes the membrane potential of neurons at the final timestep as the extracted features, thereby overlooking the temporal dynamic information, and it uses the Random-Projection encoding approach, which inherently involves information loss. As a result, to compensate for these losses, it demands the deployment of a greater number of neurons, leading to substantial increases in both area and energy consumption. Spiking-HDC incorporates temporal dynamics and notably diminishes the neuron count, but its encoding approach still suffers from inherent information loss. The proposed NeuroHDC considers temporal dynamics while eliminating any potential encoding loss; thus, it attains high accuracy using the least number of neurons. Furthermore, the encoding approach employed in NeuroHDC boasts minimal hardware complexity. These combined attributes collectively guarantee the accelerator's exceptional performance.

To further validate the superiority of our design, we additionally compare the proposed NeuroHDC accelerator with pure SNN accelerator, and detailed results are listed in Table III. For the sake of comparison fairness, we select SNN with similar model size (12.27 kB) to serve as the comparison group. The results confirm that the proposed NeuroHDC accelerator achieves performance gains in area, processing latency, and energy consumption. This performance advantage can be attributed to HDC's streamlined operational mode, centered on two core primitives: XNOR and popcount operations. By comparison, spiking neurons in SNNs are

TABLE III  
COMPARISON BETWEEN THE PROPOSED NEUROHDC-BASED ACCELERATOR AND OTHER DESIGNS

|                                      | TCAD'23 [6]<br>HyperSpikeASIC                            | ISCAS'24 [7]<br>Spiking-HDC          | This work<br>NeuroHDC-small                                                                   | This work<br>NeuroHDC-large                                     | Comparison Group<br>SNN <sup>b</sup>   |
|--------------------------------------|----------------------------------------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------|
| <b>Accuracy</b>                      | N-MNIST: 95.2%<br>DVS-Gesture: 85.3%<br>DVS-ASL: 87.8%   | N-MNIST: 94.8%<br>DVS-Gesture: 87.1% | N-MNIST: <b>97.28%</b><br>DVS-Gesture: <b>87.5%</b>                                           | DVS-ASL: <b>89.38%</b>                                          | N-MNIST: 96.45%<br>DVS-Gesture: 86.36% |
| <b>Technology</b>                    | 28nm                                                     | 40nm                                 | 130nm                                                                                         | 130nm                                                           | 130nm                                  |
| <b>Main Frequency</b>                | 500MHz                                                   | 100MHz                               | 100MHz                                                                                        | 100MHz                                                          | 100MHz                                 |
| <b>Settings</b> ( $D_{hv}/N_{max}$ ) | 4K/24                                                    | 2K/10                                | 2K/11                                                                                         | 4K/24                                                           | (16×16×2)-24-11                        |
| <b>Spiking Neurons</b>               | 32K                                                      | 200                                  | 20                                                                                            | 40                                                              | 35                                     |
| <b>Area</b>                          | 2.70mm <sup>2</sup>                                      | 0.88mm <sup>2</sup>                  | 2.494mm <sup>2</sup> ( <b>0.399mm<sup>2</sup></b> ) <sup>a</sup>                              | 7.39mm <sup>2</sup> ( <b>1.182mm<sup>2</sup></b> ) <sup>a</sup> | 2.695 mm <sup>2</sup>                  |
| <b>Inference Latency</b>             | N-MNIST: 57μs<br>DVS-Gesture: 459μs<br>DVS-ASL: 500μs    | N-MNIST: 230μs<br>DVS-Gesture: 2.2ms | N-MNIST: 80.77μs<br>DVS-Gesture: 1.8ms                                                        | DVS-ASL: 280.2μs                                                | N-MNIST: 107μs<br>DVS-Gesture: 1.84ms  |
| <b>Inference Energy</b> (μJ)         | N-MNIST: 16.30<br>DVS-Gesture: 205.61<br>DVS-ASL: 237.52 | N-MNIST: 0.45<br>DVS-Gesture: 3.74   | N-MNIST: 1.914( <b>0.315</b> ) <sup>a</sup><br>DVS-Gesture: 41.61( <b>3.01</b> ) <sup>a</sup> | DVS-ASL: 14.291( <b>1.034</b> ) <sup>a</sup>                    | N-MNIST: 2.654<br>DVS-Gesture: 46      |

<sup>a</sup> The value presented in parentheses shows the area or energy consumption scaled to the 45nm technology.

<sup>b</sup> The model size of the SNN is 12.27KB, nearly identical to that of the proposed NeuroHDC-small (12.68KB), ensuring a fair comparison.

burdened with a series of complex sequential processes, including synaptic computation, membrane potential accumulation, spike emission, and membrane potential resetting. It is the substantial disparity in hardware overhead that motivates our decision to replace the pure SNN with the hybrid SNN-HDC. Furthermore, the aforementioned SNN also exhibits a non-negligible accuracy gap. To match the accuracy achieved by the proposed NeuroHDC, a substantially larger number of spiking neurons would be required—a modification that would further exacerbate the area and energy overhead of the SNN accelerator.

In addition, relying solely on pure HDC to process event-based data proves suboptimal in practice. Although EventHD [26] achieves 96.03% (still lower than NeuroHDC) accuracy on the N-MNIST dataset, it comes with a model size of over 570.31 kB. This is because an identity hypervector must be assigned to each pixel location. As noted in Section II-B, to capture spatial correlation, these identity hypervectors need to be preconstructed; thus, their storage overhead cannot be reduced via the shift-based dynamic generation approach employed in tiny-HD [18]. In contrast, hybrid SNN-HDC exemplified by Spiking-HDC [7] first leverages one SNN layer to conduct feature extraction on raw event-based data, thus drastically cutting down the number of required base hypervectors. The proposed NeuroHDC goes a step further. It completely obviates the need for base hypervectors via its highly efficient hyperdimensional encoding scheme. Consequently, the proposed NeuroHDC achieves a model size of merely 12.68 kB, encompassing SNN's weight and class hypervectors, which is significantly smaller than that of EventHD. Despite the absence of EventHD's hardware implementation, the model size comparison enables us to infer that NeuroHDC will feature a considerably smaller chip area, given that SRAM macros dominate the area of storage-intensive hardware designs. Accordingly, the NeuroHDC chip can be expected to possess lower manufacturing costs and reduced power consumption.

TABLE IV  
PERFORMANCE OF THE PROPOSED ACCELERATOR ON FPGA

| Approach                       | NeuroHDC-small                        | NeuroHDC-large  |
|--------------------------------|---------------------------------------|-----------------|
| LUTs                           | 4.9k                                  | 9.7k            |
| FFs                            | 3.8k                                  | 5.3k            |
| RAMs                           | 21                                    | 41              |
| DSPs                           | 0                                     | 0               |
| Power(W)                       | 0.268                                 | 0.292           |
| Throughput (SPS <sup>a</sup> ) | N-MNIST: 1570.9<br>DVS-Gesture: 321.7 | DVS-ASL: 1191.2 |

<sup>a</sup> Samples per second.

### C. Evaluation on FPGA

The proposed accelerator is also deployed and verified on the FPGA platform, achieved by substituting the internal SRAM macro with the readily available BRAM resource in the FPGA. The resource usage and measured performance of the accelerator are listed in Table IV. In addition, considering that almost all the existing designs are based on ASIC, achieving performance evaluation and comparison necessitates the completion of intricate front-end and back-end design processes. Hence, we aspire for the FPGA-based evaluation data presented in this article to establish a foundational baseline, enabling researchers in this area to swiftly undertake comparative analyses.

### D. Discussion

This article introduces NeuroHDC, a novel framework that achieves synergistic integration of SNN and HDC through the efficient, lossless hyperdimensional encoding. Designed for event-based data obtained by DVS, NeuroHDC enables high-efficient inference and transfer learning capabilities. For hardware implementation, we have developed a custom ASIC-based accelerator tailored to NeuroHDC, which is capable of processing AER format data output by any DVS.

Recently, emerging processing-in-sensor (PIS) designs incorporate either spiking neuron circuits [41] or HDC modules [42] within the sensor itself, significantly cutting down the data transmission overhead to the accelerator. In fact, the proposed NeuroHDC is fundamentally orthogonal to the PIS paradigm. This orthogonality enables a potential PIS-based implementation of NeuroHDC, which would achieve higher integration and lower energy consumption than our ASIC-based one. This performance gain, however, comes at the cost of generality, as the PIS-based one would be inherently specialized and tied to its integrated sensing array. Consequently, we consider this outside the scope of our current investigation and leave it to future work.

## VII. CONCLUSION

In this article, we introduce NeuroHDC, a neuromorphic hyperdimensional computing (HDC) method, alongside an innovative hyperdimensional encoding technique characterized by its low complexity and absence of information loss. Owing to the aforementioned features of NeuroHDC and the proposed training scheme, it achieves superior accuracy while utilizing much fewer neurons. Leveraging the remarkable efficiency and lightweight nature of NeuroHDC, we further propose a dedicated accelerator design that delivers significant area savings exceeding 50%, together with energy consumption reductions ranging from 20% to 90%.

## REFERENCES

- [1] H. AliAkbarpour, A. Moori, J. Khorramdel, E. Blasch, and O. Tahri, "Emerging trends and applications of neuromorphic dynamic vision sensors: A survey," *IEEE Sensors Rev.*, vol. 1, pp. 14–63, 2024.
- [2] L. Cordone, B. Miramond, and S. Ferrante, "Learning from event cameras with sparse spiking convolutional neural networks," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8.
- [3] C. Teeter et al., "Generalized leaky integrate-and-fire models classify multiple neuron types," *Nature Commun.*, vol. 9, no. 1, p. 709, Feb. 2018.
- [4] Y. Kuang et al., "A 64K-neuron 64M-1b-synapse 2.64pJ/SOP neuromorphic chip with all memory on chip for spike-based models in 65nm CMOS," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 7, pp. 2655–2659, Jul. 2021.
- [5] J. Morris et al., "HyperSpike: Hyperdimensional computing for more efficient and robust spiking neural networks," in *Proc. Design, Autom. Test Europe Conf. Exhib. (DATE)*, 2022, pp. 664–669.
- [6] T. Zhang et al., "HyperSpikeASIC: Accelerating event-based workloads with HyperDimensional computing and spiking neural networks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 11, pp. 3997–4010, Nov. 2023.
- [7] A. Xiao, X. Zhang, J. Yang, L. Zheng, and Z. Zou, "Spiking-HDC: A spiking neural network processor with HDC classifier enabling transfer learning," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2024, pp. 1–5.
- [8] P. Kanerva, "Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors," *Cognit. Comput.*, vol. 1, no. 2, pp. 139–159, Jun. 2009.
- [9] L. Ge and K. K. Parhi, "Classification using hyperdimensional computing: A review," *IEEE Circuits Syst. Mag.*, vol. 20, no. 2, pp. 30–47, 2nd Quart., 2020.
- [10] T. Yu, B. Wu, K. Chen, G. Zhang, and W. Liu, "Fully learnable hyperdimensional computing framework with ultratiny accelerator for edge-side applications," *IEEE Trans. Comput.*, vol. 73, no. 2, pp. 574–585, Feb. 2024.
- [11] W. Fang, Z. Yu, Y. Chen, T. Masquelier, T. Huang, and Y. Tian, "Incorporating learnable membrane time constant to enhance learning of spiking neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 2661–2671.
- [12] D. Hebb, *The Organization of Behavior: A Neuropsychological Theory*. London, U.K.: Lawrence Erlbaum Associates, 2002.
- [13] G.-Q. Bi and M.-M. Poo, "Synaptic modifications in cultured hippocampal neurons: Dependence on spike timing, synaptic strength, and postsynaptic cell type," *J. Neurosci.*, vol. 18, no. 24, pp. 10464–10472, Dec. 1998.
- [14] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, "Conversion of continuous-valued deep networks to efficient event-driven networks for image classification," *Frontiers Neurosci.*, vol. 11, p. 682, Dec. 2017.
- [15] Y. Jin, W. Zhang, and P. Li, "Hybrid macro/micro level backpropagation for training deep spiking neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 7005–7015.
- [16] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-temporal backpropagation for training high-performance spiking neural networks," *Frontiers Neurosci.*, vol. 12, p. 331, May 2018.
- [17] T. Yu, B. Wu, K. Chen, G. Zhang, and W. Liu, "LAHDC: Logic-aggregation-based query for embedded hyperdimensional computing accelerator," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 1, pp. 119–129, Jan. 2025.
- [18] B. Khaleghi, H. Xu, J. Morris, and T. Š. Rosing, "Tiny-HD: Ultra-efficient hyperdimensional computing engine for IoT applications," in *Proc. Design, Autom. Test Europe Conf. Exhib. (DATE)*, 2021, pp. 408–413.
- [19] M. Imani, J. Messerly, F. Wu, W. Pi, and T. Rosing, "A binary learning framework for hyperdimensional computing," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 126–131.
- [20] A. Rahimi, P. Kanerva, and J. M. Rabaey, "A robust and energy-efficient classifier using brain-inspired hyperdimensional computing," in *Proc. Int. Symp. Low Power Electron. Design*, 2016, pp. 64–69.
- [21] T. Yu, B. Wu, K. Chen, G. Zhang, and W. Liu, "Memristor-based approximate query architecture for in-memory hyperdimensional computing," *IEEE Trans. Comput.*, vol. 73, no. 11, pp. 2605–2618, Nov. 2024.
- [22] A. Hernández-Cano, Y. Kim, and M. Imani, "A framework for efficient and binary clustering in high-dimensional space," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1859–1864.
- [23] T. Yu, B. Wu, K. Chen, G. Zhang, and W. Liu, "AttnACQ: Attentioned-AutoCorrelation-based query for hyperdimensional associative memory," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 12, pp. 4984–4988, Dec. 2024.
- [24] M. Imani et al., "QuanthD: A quantization framework for hyperdimensional computing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2268–2278, Oct. 2020.
- [25] S. Duan, Y. Liu, S. Ren, and X. Xu, "LeHDC: Learning-based hyperdimensional computing classifier," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 1111–1116.
- [26] Z. Zou, H. Alimohamadi, Y. Kim, M. H. Najafi, N. Srinivasa, and M. Imani, "EventHD: Robust and efficient hyperdimensional learning with neuromorphic sensor," *Frontiers Neurosci.*, vol. 16, pp. 1–14, Jul. 2022.
- [27] A. Mitrokhin, P. Sutor, C. Fermüller, and Y. Aloimonos, "Learning sensorimotor control with neuromorphic sensors: Toward hyperdimensional active perception," *Sci. Robot.*, vol. 4, no. 30, p. 6736, May 2019.
- [28] N. Anwar, C. Parameshwara, C. Fermüller, and Y. Aloimonos, "Towards an improved hyperdimensional classifier for event-based data," in *Proc. 57th Annu. Conf. Inf. Sci. Syst. (CISS)*, Mar. 2023, pp. 1–4.
- [29] E. Hassan et al., "Efficient event-based robotic grasping perception using hyperdimensional computing," *Internet Things*, vol. 26, Jul. 2024, Art. no. 101207.
- [30] Z. Zou et al., "Memory-inspired spiking hyperdimensional network for robust online learning," *Sci. Rep.*, vol. 12, no. 1, p. 7641, May 2022.
- [31] A. Xiao et al., "A 40-nm sub-mJ/transfer HDC-SNN hybrid processor enabling on-chip few-shot transfer learning for IoT applications," *IEEE Trans. Circuits Syst. Artif. Intell.*, early access, Jan. 28, 2026, doi: 10.1109/TCASAI.2026.3658593.
- [32] A. G. Ayar, A. Maida, and M. Margala, "HyperEncoding: Spiking neural networks with hyperdimensional encoding for robust edge intelligence," in *Proc. Great Lakes Symp. VLSI*, Jun. 2025, pp. 91–96.
- [33] L. Zhou, H. Wang, B. Wang, J. Wang, Z. Yu, and S. Xiao, "SynapseHD: A unified training framework for bridging spiking neural networks and hyperdimensional computing," *Neurocomputing*, vol. 659, Jan. 2026, Art. no. 131757.
- [34] C. Kinavuidi, L. Peres, and O. Rhodes, "Hyperdimensional decoding of spiking neural networks," *Neuromorphic Comput. Eng.*, vol. 6, no. 1, Mar. 2026, Art. no. 014021.

- [35] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, Nov. 2019.
- [36] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2704–2713.
- [37] G. Orchard, A. Jayawant, G. K. Cohen, and N. Thakor, "Converting static image datasets to spiking neuromorphic datasets using saccades," *Frontiers Neurosci.*, vol. 9, p. 437, Nov. 2015.
- [38] A. Amir et al., "A low power, fully event-based gesture recognition system," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 7243–7252.
- [39] Y. Bi, A. Chadha, A. Abbas, E. Bourtsoulatz, and Y. Andreopoulos, "Graph-based object classification for neuromorphic vision sensing," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 491–501.
- [40] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm," *Integration*, vol. 58, pp. 74–81, Jun. 2017.
- [41] L. Chen et al., "A brain-inspired intelligent vision sensor with spiking processing-in-sensor architecture for spike feature encoding," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 72, no. 10, pp. 1368–1372, Oct. 2025.
- [42] X. Huang et al., "SenHDC: A 3-D NAND flash-based processing-in-sensor hyperdimensional computing architecture," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 7, pp. 1899–1909, Jul. 2025.



**Tianyang Yu** (Graduate Student Member, IEEE) received the B.Sc. degree from the College of Electronic and Information Engineering and College of Integrated Circuits, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2022, where he is currently working toward the Ph.D. degree in information and communication engineering.

His current research interests include hardware architecture design for AI algorithms, approximate computing, in-memory computing, neuromorphic computing, and hyperdimensional computing.



**Bi Wu** (Member, IEEE) received the Ph.D. degree from the School of Electronic Information Engineering, Beihang University, Beijing, China, in 2020.

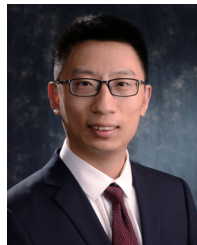
In 2019, with the financial support of China Scholarship Council, he spent one year as a Visiting Graduate Student at the University of Notre Dame, Notre Dame, IN, USA, under the supervision of Prof. Xiaobo Sharon Hu. After his Ph.D. degree, he joined the College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, as an Assistant Professor. His research interests include magnetic memory architecture, spintronic device-based in-memory computing architecture, and neural network accelerator design.



**Ke Chen** (Member, IEEE) received the B.Sc. degree in engineering from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2010, and the Ph.D. degree in computer engineering from Northeastern University, Boston, MA, USA, in 2020.

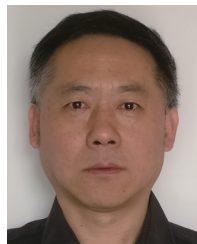
He is currently an Associate Researcher at the College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China. His research interests include low-power and high-performance VLSI designs, emerging logic and memory devices and circuits, and approximate and fault-tolerant computing.

Dr. Chen is a member of the IEEE CAS VLSI Systems and Applications Technical Committee.



**Chenggang Yan** (Member, IEEE) received the B.Sc. degree in electronics from Northwestern Polytechnical University, Xi'an, China, in 2012, and the M.Sc. and Ph.D. degrees in microelectronics from Southeast University, Nanjing, China, in 2014 and 2020, respectively.

In 2020, he was an Assistant Professor at Nanjing University of Aeronautics and Astronautics, Nanjing. His current research interests include frequency synthesizer designs, integrated circuits for wireless communications, and approximate digital baseband circuit designs.



**Gong Zhang** (Senior Member, IEEE) received the Ph.D. degree in electronic engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2002.

From 1990 to 1998, he was a Member of Technical Staff at No. 724 Institute China Shipbuilding Industry Corporation (CSIC), Nanjing. Since 1998, he has been with the College of Electronic and Information Engineering at NUAA, where he is currently a Professor. His research interests include signal processing for communications, radar, and sensor processing; statistical and array signal processing; applications of linear algebra and optimization methods in signal processing and communications; estimation and detection theory; sampling theory; and cooperative and cognitive systems.

Dr. Zhang is a member of the Committee of Electromagnetic Information, Chinese Society of Astronautics (CEI-CSA), a member of the Editorial Board of *Journal of Radar*, China, and a Senior Member of Chinese Institute of Electronics.



**Weiqiang Liu** (Senior Member, IEEE) received the B.Sc. degree in information engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2006, and the Ph.D. degree in electronic engineering from Queen's University Belfast (QUB), Belfast, U.K., in 2012.

He is currently a Professor and the Dean of the College of Integrated Circuits, NUAA. He has authored two research books and over 200 leading journals and conference papers. His research interests focus on energy-efficient and secure computing

circuits and systems.

Dr. Liu was awarded the prestigious Excellent Young Scholar Award by NSFC in 2020. He serves as the VP-Elect for the IEEE Nanotechnology Council, the Steering Committee Chair of IEEE TVLSI, an Associate Editor for IEEE TC, TCAS-I, and TETC, and a Guest Editor of PROCEEDINGS OF THE IEEE. He is the Program Co-Chair of IEEE ARITH 2020, ACM NANOARCH 2022, and IEEE AsianHOST 2023.